

BANKAI FINOPS SANDBOX & PORTFOLIO SHOWCASE

Continuous Financial Testing, LLM Resilience & Interview Showcase for AI QA Architects

Why this Sandbox was engineered: In tier-1 commercial and investment banking, hiring an AI Test Architect requires far more than theoretical knowledge. Candidates must tangibly demonstrate the ability to **model unit inference costs down to fractions of a cent**, **detect silent financial failovers**, and **thwart Denial of Wallet attacks**. This autonomous banking sandbox operates at **zero cost** (no paid API keys required, deterministic and stochastic mathematical emulation) and serves as an **interview-ready engineering showcase hosted on GitHub**.

Core Sandbox Pillar	Operational Simulation Role	Career & Portfolio Value
1. High-Fidelity Engine	Simulates on-prem sovereign vLLM (Llama-3-70B), GPT-4o, and Claude 3.5 Sonnet with stochastic token arithmetic and true prefix caching detection (prefixes >= 1024 tokens).	Demonstrates deep command of token economics, TTFT, KV caching, and inference hosting architectures.
2. Financial Chaos Lab	Injects 4 production failure modes: dynamic cache-busting, silent public cloud failover spills, KYC Denial of Wallet bombs, and runaway autonomous agent loops.	Proves engineering mastery in designing software circuit breakers and non-functional resilience tests.
3. Visual Reporting Suite	Interactive HTML5 dashboard (Chart.js), terminal telemetry cards, automated Pytest test suite (5 assertions), and markdown case study generator.	Provides tangible, visually compelling deliverables ready to showcase live to hiring managers.

Banking Inference Pipeline & QA Control Gates Architecture



PUBLIC GITHUB REPOSITORY DEPLOYMENT

This repository is published at <https://github.com/jeanyvesgarcin/bankai-finops-sandbox>. It is fully configured with MIT License, Pytest test suites, Chart.js HTML dashboards, and comprehensive documentation.

2. Detailed Breakdown of the 3 Banking Use Cases

The module engine/workloads.py models 3 realistic tier-1 banking systems subject to strict regulatory mandates and tight unit cost constraints:

Use Case 1: Retail Credit Decisioning & Scoring (Basel IV / EBA / ACPR)

- **Business Scope:** Classified as an **EU AI Act Annex III High-Risk System**. Underwrites personal loans, computes debt-to-income (DTI cap of 35.00%), and calculates Probability of Default (PD).
- **FinOps Characteristic:** The regulatory prompt contains over **2,500 tokens** of immutable legal requirements. When Prefix Caching is maintained, prompt tokens enjoy a 50% to 90% discount. Nominal cost per credit decision: **< 0.0004 €**.
- **QA Risk:** Any accidental alteration of the system prompt prefix resets cache for 100% of downstream traffic.

Use Case 2: Corporate KYC & Financial Statement Extraction (5th AML Directive)

- **Business Scope:** Automated compliance analysis of annual filings, balance sheets, and certificates of incorporation. Identifies Ultimate Beneficial Owners (UBO > 25%) and screens against Politically Exposed Persons (PEP) registries.
- **FinOps Characteristic:** Input payloads vary dramatically (from 2 pages to 500-page consolidated filings).
- **QA Anti-DoW Guardrail:** A pre-execution gate inspects page counts and byte sizes. Payloads exceeding 10 pages or 50 KB are intercepted before LLM invocation, **achieving 100% cost avoidance (0.00 € spent)**.

Use Case 3: Private Banking Wealth Advisory Copilot (MiFID II Compliant)

- **Business Scope:** AI assistant for private wealth bankers tailoring multi-asset portfolios for High Net Worth Individuals (HNWI). Long conversations accumulate historical conversational context.
- **FinOps Characteristic:** Without guardrails, a 10-turn session spirals from 1,000 tokens to over 12,000 tokens per turn (quadratic cost inflation).
- **QA Sliding Window Guardrail:** Retains the system prompt plus a compressed executive memo of previous turns, keeping context flat and predictable.

Banking Workload	Inference Engine	Tokens / Request	NFR FinOps Cap	QA Guardrail Enforced
Retail Credit Scoring	Llama-3-70B / GPT-4o-mini	2,800 tokens (80% cached)	CPIT < 0.0005 €	Assertion Cache Hit Ratio > 75%
Corporate KYC Extraction	GPT-4o / Vision OCR	5,000 to 45,000 tokens	Reject if doc > 10 pages	Pre-Execution Anti-DoW Filter
Private Wealth Copilot	Claude 3.5 / GPT-4o-mini	1,500 constant tokens	Session cost < 0.01 €	Sliding Context Window (k=2)

3. Financial Chaos Lab & Anomaly Injection

In chaos/chaos_injector.py, you can programmatically inject 4 real-world production failures. In technical job interviews, this is your strongest differentiator: demonstrating not only that systems can fail, but exactly how your QA guardrails prevented catastrophic budget blowouts.

Financial Anomaly	Unprotected Production Hazard	Automated QA Guardrail	Net Savings
1. Cache-Busting (Cache Destruction)	A developer prepends a dynamic UUID or timestamp to the system prompt. The model provider treats every call as a brand-new prompt: cache hit drops to 0%, tripling costs.	Automated Pytest regression gate verifying cache_hit_ratio and static linter forcing dynamic variables to payload tail.	+34.5% savings
2. Silent Fallback (Uncapped Cloud Spill)	On-prem sovereign vLLM suffers Out-Of-Memory (OOM) crashes. Traffic silently diverts to OpenAI GPT-4o Cloud. 10,000 req/hour leak to expensive public APIs (violating DORA).	FallbackCircuitGuard on sliding window: trips after 10 fallback requests and switches to deterministic rule fallback (0.00 €).	+73.4% savings
3. Denial of Wallet (KYC Bombardment)	A malicious bot or misconfigured ETL pipeline pushes 50-page financial audits (350 KB). The bank is billed for massive vision/LLM token consumption.	Pre-execution gate rejecting 100% of oversized documents before model invocation, logging a security audit alert.	+100.0% (0€ spent)
4. Runaway Agent (Infinite Tool Loop)	An autonomous AML investigation agent gets trapped in circular corporate shareholding exploration (40 recursive iterations) due to ambiguous stopping criteria.	MaxStepBudgetGuard enforcing a hardware-like cutoff at step 5 with automatic escalation to human compliance officers.	+87.5% savings

Consolidated Benchmark Financial Summary

Consolidated KPI	Unprotected Baseline (Naive)	With QA FinOps Guardrails	Net Financial ROI
Raw Inference Spend	4.8402 €	0.1631 €	+4.6771 € (96.6%)
Monthly Extrapolation (100k req)	4,840.20 € / month	163.10 € / month	+4,677.10 € / month
Regulatory Governance Status	REJECTED (Non-Compliant)	APPROVED FOR PRODUCTION (CAB)	Audit Score: 100/100 (Grade A)

THE EXECUTIVE ELEVATOR PITCH

“Across a test sample, implementing 4 automated QA guardrails reduced inference expenditures by 96.6%. At enterprise scale across 500,000 monthly credit applications, this guarantees over 23,000 € in recurring monthly cost savings.”

4. Step-by-Step Execution Guide & Visual Tools

The sandbox is fully orchestrated from the terminal using the automation script `sandbox.ps1`. Here is how to run each component and observe live results:

Action 1: Run Full Simulation & Chaos Suite

1-CLICK RUN COMMAND

In the terminal, run:
`.\sandbox.ps1 run`
(Or directly: `python run_sandbox.py --mode all`)

What happens: The engine processes 40 credit decisions, 15 KYC extractions, and 6 wealth turns, computes live metrics, injects the 4 chaos failure modes, renders the HTML dashboard, and refreshes `PORTFOLIO_CASE_STUDY.md`.

Action 2: Open Interactive Visual Dashboard

OPEN WEB DASHBOARD

Run: `.\sandbox.ps1 html`
Your default browser opens `bankai_finops_dashboard.html` instantly.

What you see on screen:

- **Header KPI Cards:** Net Savings (+4.68 €), Prefix Caching Ratio (70.2%), Average Cost per Decision (0.00078 €).
- **Chart.js Bar Chart:** Direct visual comparison between 'Unprotected Baseline (Red)' vs 'QA Guardrails (Green)'.
- **Doughnut Breakdown:** Distribution of cost savings by chaos scenario.
- **Detailed Audit Matrix:** Technical remediation actions implemented by the QA Architect.

Action 3: Execute Automated Validation Tests (Pytest)

AUTOMATED TEST EXECUTION

Run: `.\sandbox.ps1 test`
Output: **5 passed in ~0.19s**. All financial invariants and circuit breakers are confirmed green.

What the 5 automated assertions validate:

1. `test_credit_scoring_nominal_budget`: Asserts unit cost (< 0.0005 €) and verifies prefix cache hit.
2. `test_cache_busting_anomaly_mitigation`: Verifies cache retention delivers >30% cost savings.
3. `test_silent_fallback_circuit_breaker`: Confirms circuit breaker trips on unauthorized cloud failovers.
4. `test_denial_of_wallet_guard_blocks_heavy_docs`: Confirms 100% rejection of oversized filings at 0€ cost.
5. `test_wealth_copilot_sliding_window_inflation`: Proves context truncation halts quadratic budget inflation.

5. Portfolio Showcase & Technical Interview Strategy

This project is your **flagship engineering showcase**. Here is the exact playbook to present it with the posture of an elite AI Test Architect:

1. GitHub Repository Configuration

- **Repository URL:** <https://github.com/jeanyvesgarcin/bankai-finops-sandbox>
- **Repository Description:** *'High-fidelity banking AI FinOps & resilience lab. Simulates LLM cost drift, prompt caching, denial of wallet attacks, and circuit breakers for regulated tier-1 banks (EU AI Act & DORA compliant).'*
- **Configured Topics:** ai-finops, qa-architecture, llm-evaluation, banking-ai, dora-compliance, pytest.
- **Flagship Deliverable:** Showcase PORTFOLIO_CASE_STUDY.md, highlighting architecture diagrams and audit metrics.

2. Resume & LinkedIn Profile Positioning

RECOMMENDED CV PROJECT ENTRY

AI Test Architect & FinOps Lead — Apex Bank AI FinOps Framework (Open Source)

- Engineered continuous financial testing framework for LLMs in regulated banking (Basel IV, DORA Art. 28).
- Delivered 96.6% inference cost reduction under chaos conditions via Prompt Caching and software circuit breakers.
- Built pre-execution Denial of Wallet filters and blocking CI/CD Quality Gates (Audit Score: 100/100 Grade A).

3. The Technical Interview STAR Pitch Script

STAR Stage	Exact Pitch Script to Recruiter / Engineering Director
Situation	"At Apex Bank, delivery squads were deploying Generative AI models without unit cost visibility, risking massive cloud budget blowouts if sovereign on-prem clusters degraded."
Task (QA Scope)	"As AI Test Architect, my objective was to define Non-Functional Financial Requirements (NFRs), build automated testing gates, and guarantee DORA Art. 28 compliance before production sign-off."
Action	"I architected a banking sandbox simulating 3 financial workflows and injecting 4 production chaos modes (cache-busting, public cloud spills, Denial of Wallet bombs, and runaway agent loops)."
Quantified Result	"My test suite proved a 96.6% cost reduction during crisis scenarios, capped transaction costs below 0.0005 €, and achieved 100/100 CAB audit compliance."

6. Quick Reference Cheat Sheet & Training Checklist

Keep this page accessible during personal practice sessions and prior to technical interviews:

CLI Command Quick Reference

PowerShell Command	Operational Action Executed	Output Artifacts Produced
.\sandbox.ps1 run	Executes nominal banking traffic + 4 chaos scenarios, computes metrics and exports reports.	Console telemetry + bankai_finops_dashboard.html + PORTFOLIO_CASE_STUDY.md
.\sandbox.ps1 test	Executes the 5 automated Pytest assertions verifying budget caps and circuit breakers.	Standard Pytest output (5 passed in ~0.19s)
.\sandbox.ps1 html	Opens the interactive Chart.js dashboard directly in your default web browser.	Renders bankai_finops_dashboard.html
.\sandbox.ps1 portfolio	Regenerates the recruiter-ready GitHub markdown report with updated execution metrics.	Refreshes PORTFOLIO_CASE_STUDY.md

The 2 Mandatory Mathematical Formulas to Memorize

1. Cost per Inference Transaction (CPIT)	$CPIT = (Tokens_{in_uncached} * P_{in} + Tokens_{in_cached} * P_{cached} + Tokens_{out} * P_{out}) / 10^6$
2. On-Prem GPU Break-Even Point	$Monthly_Break_Even = Amortized_Monthly_CapEx / (CPIT_{Public_Cloud} - CPIT_{Sovereign_Local})$

6-Step Hands-On Practice Checklist

- [] Step 1: Open your terminal in bankai_finops_sandbox/ and run .\sandbox.ps1 run.
- [] Step 2: Open the visual dashboard via .\sandbox.ps1 html and inspect the 2 interactive Chart.js graphs.
- [] Step 3: Run .\sandbox.ps1 test to verify that all 5 financial assertions pass green.
- [] Step 4: Review PORTFOLIO_CASE_STUDY.md to absorb tier-1 banking terminology.
- [] Step 5: Practice reciting the STAR Pitch (page 5) out loud in under 60 seconds.
- [] Step 6: Link <https://github.com/jeanyvesgarcin/bankai-finops-sandbox> directly on your LinkedIn & CV.

YOU ARE 100% ARMED FOR YOUR NEXT TIER-1 BANKING ROLE

With this sandbox, you are no longer a theoretical QA engineer: you are an **operational AI Test Architect** equipped to safeguard banking balance sheets and lead AI FinOps conversations from Day 1!